B.6 Lab 6: Iterative Methods for Linear System Solves

Problem 1: The Heat Equation and Componentwise Splitting Methods

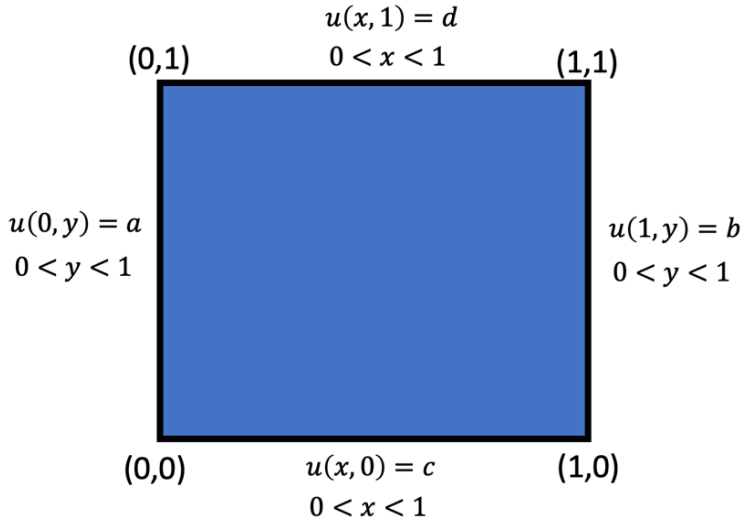
The heat equation is a partial differential equation that describes the flow of heat through a thermally conductive material (under ideal assumptions such as uniform density, no internal heat sources). The function $u(x, y, t)$ is a solution to the heat equation if

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

where α is a positive constant called the thermal diffusivity of the medium. The equation has a rich history and is important in many applications in physics and mathematical modeling.[13] While the heat equation describes the flow of heat as a function of time, we are often interested in how the temperature ultimately settles, or in other words, what is the temperature distribution when temperature stops varying with respect to time:

$$0 = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right).$$

This condition is called steady-state and the corresponding equation is called Laplace's Equation. Let's consider a square sheet of thermal conducting material and assume that we can control the temperature on the edges:



These "controls" are referred to as boundary conditions. One approach for computing a numerical solution to Laplace's equation is to employ finite differences to approximate the partial derivatives on a uniform grid (this is called a "discretization"). Doing so results in a linear system of equations that describes the solution u at each discretized point on the grid. Download the function `laplacianMat.m` and take a look at the header. We can use `laplacianMat` and MATLAB's `\` to solve the Laplacian for the boundary conditions $a = 0$, $b = 1$, $c = 0$, $d = 1$:

```
>> [A,b,D] = laplacianMat([0 1 0 1]);
>> x = A\b; n = length(D); U = zeros(n,n);
>> U(:) = x; close all; figure(1); surf(D,D,U)
```

[13] https://en.wikipedia.org/wiki/Heat_equation

Once you are confident that you understand the above lines of code, try experimenting with the boundary conditions and see how they change the steady state solution. In this lab, we are going to solve the Laplacian using matrix splitting methods.

Chapter 5 implements the splitting methods in Listing 5.2 by forming the splitting matrices M and N and repeatedly solving against M with the backslash operator. That implementation is the clearest way to see the mathematics, but it is not how these methods are deployed at scale. Forming M and N doubles the required storage for no benefit, and for the enormous sparse systems described in Chapter 5, there is no memory to spare for either. In this lab, we will implement all three splitting methods the way production codes do: by sweeping directly on the entries of A, one row at a time, without ever forming M or N and without ever calling the backslash operator.

Problem 1.1: Write your own function `myJacobi.m` with the following signature:

```
function [x, iter, resHist] = myJacobi(A, b, x0, tol, maxIter)
```

The function applies the componentwise Jacobi update, Equation (5.6), to solve $A\mathbf{x} = \mathbf{b}$ starting from the initial guess `x0`. Each sweep should compute the update for component i directly from row i of A, using the row segments to the left and right of the diagonal, either as vectorized products (for example, `A(i,1:i-1)*x(1:i-1)`) or with an inner for-loop. Since Jacobi reads only old values, your code will need two vectors, one holding the current iterate and one receiving the new. The stopping criteria should be defined by the relative residual:

$$\frac{\|A\mathbf{x}_k - \mathbf{b}\|}{\|\mathbf{b}\|} < \texttt{tol}.$$

The output `iter` is the number of sweeps performed, and `resHist` is a vector containing the relative residual after each sweep. If the tolerance is not met within `maxIter` sweeps, the function should stop and return its current iterate. Be sure your code checks that the inputs supplied to the function have the correct dimensions.

To validate your function, apply it to the worked example of Chapter 5,

$$\begin{bmatrix} 4 & -1 & 1 \\ -2 & 6 & 1 \\ -1 & 1 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 9 \\ -6 \end{bmatrix}.$$

Starting from $\mathbf{x}_0 = (0, 0, 0)$ with $\texttt{tol} = 10^{-6}$, your function should converge to $(1, 2, -1)$ in 12 iterations, and your `resHist` should match the Jacobi column of Table 5.1.

Problem 1.2: Write your own function `myGaussSeidel.m` with the same signature as `myJacobi`. The Gauss-Seidel sweep is nearly identical to the Jacobi sweep, with one crucial difference: there is only one solution vector, updated in place. By the time your loop reaches row i, entries 1 through $i-1$ of `x` already hold their new values, so the same row segment product now automatically reads the updated components. Be sure you understand why the same line of code behaves differently in the two methods before you start writing. Validate on the worked example above: starting from $\mathbf{x}_0 = (0, 0, 0)$ with $\texttt{tol} = 10^{-6}$, your function should converge in 6 iterations and reproduce the Gauss-Seidel column of Table 5.1.

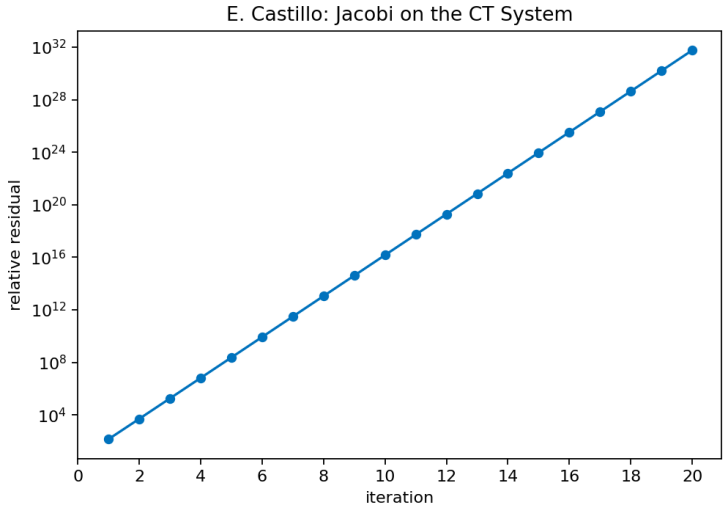
Problem 1.3: Write your own function `mySOR.m` with the signature:

```
function [x, iter, resHist] = mySOR(A, b, x0, tol, maxIter, lambda)
```

Chapter 5 presents SOR only in its splitting form, Equation (5.13). Your first task is to derive the componentwise update by applying the relaxation formula, Equation (5.12), to a single component of the Gauss-Seidel sweep. Write the resulting update formula in your function header, then implement it. As a sanity check, `lambda` $= 1$ must reproduce your `myGaussSeidel` results exactly, iteration counts and all.

Problem 2: Not Every System Cooperates

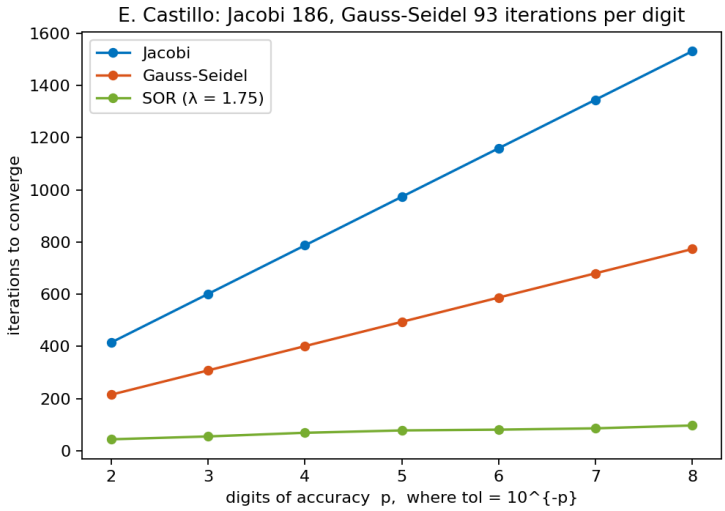
Unlike Gaussian elimination, the splitting methods are not guaranteed to converge for every linear system. Write a script `jacobiCT.m` that loads the CT reconstruction system from Lab 5 (`CTdata40x40.mat`) and applies your `myJacobi` with `x0 = b`, `tol` $= 10^{-5}$, and `maxIter` $= 20$. Use `semilogy` to plot the relative residual history. It should look something like this:



The iteration is diverging, with the residual growing by roughly a fixed multiple every sweep. Your script should also determine why. Add code that computes, for each row of a matrix, the magnitude of the diagonal entry and the sum of the magnitudes of the off-diagonal entries, and reports how many rows satisfy the strict diagonal dominance inequality of Theorem 5.1. Run this check on both the CT matrix and the heat equation matrix from Problem 1. In your header, report the two counts and use them, along with Theorem 5.1, to explain the two behaviors. Note the subtlety on the heat equation side: most of its rows satisfy the dominance comparison only with equality, so the theorem technically does not apply, and yet the methods converge. A sufficient condition is not a necessary one. Finally, in a sentence, explain why Lab 5 attacked the CT system with elimination while this lab attacks the heat system with iteration.

Problem 3: The Price of Accuracy

How does the cost of an iterative solve grow with the accuracy demanded? Write a script `accuracyStudy.m` that solves the heat equation system `[A,b,D] = laplacianMat([0 1 0 1])` using all three of your methods, with `x0 = b`, for each tolerance `tol` $= 10^{-p}$ with $p = 2, 3, \ldots, 8$. For SOR, use $\lambda = 1.75$, the optimal value identified in Figure 5.7. As a check on your solvers, at `tol` $= 10^{-5}$ your iteration counts should agree with the values reported in Chapter 5 for this system. Generate a single plot showing the number of iterations as a function of the digits of accuracy p, with one labeled curve per method. It should look something like this:



State the cost per digit for Jacobi and Gauss-Seidel in the title of the plot. In your header, describe the pattern: each method pays a fixed number of iterations for every additional digit of accuracy, and the fixed number is a property of the method, not of the tolerance. Where does that number come from? We will find out in Chapter 6.

General Submission Guidelines

```
[FirstName][LastName]_[filename].m            For programs
[FirstName][LastName]_[filename]_plot_1.pdf   For corresponding plots (export them to pdf)
[FirstName][LastName]_[filename]_plot_2.pdf   For problem with more than one plot
```

Submission Checklist

- Problem 1.1: `myJacobi.m`
  - Complete header and commented code
  - Correct componentwise sweep using row indexing, with no M, N, or backslash
  - Correct stopping criteria, `maxIter` guard, and `resHist` output
  - Validation against the worked example (12 iterations)
- Problem 1.2: `myGaussSeidel.m`
  - Complete header and commented code
  - Correct in-place sweep
  - Validation against the worked example (6 iterations)
- Problem 1.3: `mySOR.m`
  - Header containing the derived componentwise SOR update
  - Correct implementation
  - `lambda` $= 1$ reproduces the Gauss-Seidel results
- Problem 2: `jacobiCT.m`

  - Correct `semilogy` divergence plot
  - Correct diagonal dominance check for both matrices
  - Header explanation of the two behaviors using Theorem 5.1

- Problem 3: `accuracyStudy.m`
  - Correct tolerance sweep for all three methods
  - Correct plot with labeled curves and cost per digit in the title
  - Header description of the cost-per-digit pattern